

FPGA -BASED Serial Communications and Synchronization Protocol for Front and Rear Line-Scan Cameras

Name

Devendra Singh

Date

27/08/2026

FPGA/device used: Intel Cyclone V -5CGXFC7C7F23C8

Tool: Quartus Prime 25.1 Standard Lite

Simulation: Icarus Verilog /Questa

Project Overview:

The project implements a 63-bit digital data communication and processing path using Verilog RTL. The design accepts parallel input data, converts the data into a serial stream, transfers the serialized data through a modeled serial link, reconstructs the original parallel data at the receiver, transfers the received data through a handshake-based Clock Domain Crossing (CDC) interface, and finally processes the transferred data using a pixel-processing block.

The design is divided into multiple functional RTL blocks including Serializer, Serial Link, Deserializer, CDC Handshake, and Pixel Processor. Each block has a defined interface and control mechanism, allowing the complete system to be developed and verified incrementally.

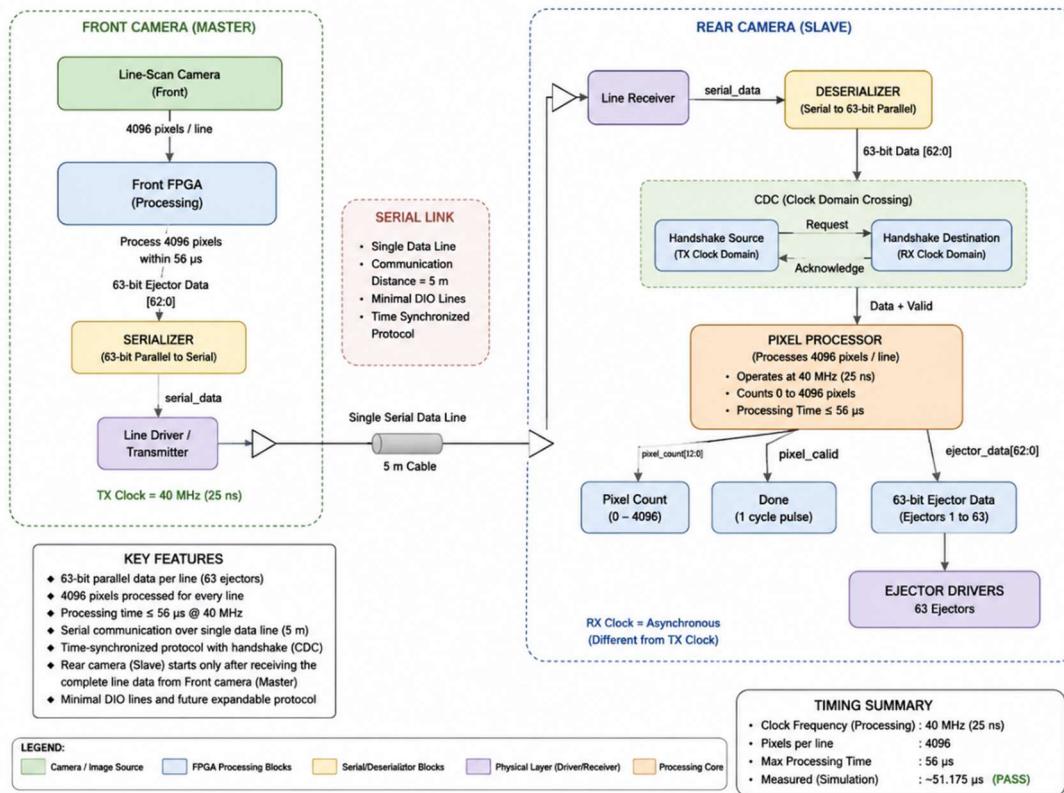
Functional verification was performed using multiple input data patterns and different clock periods. The final testbench verifies serial data recovery, CDC data integrity, and pixel-processing completion

Problem Statement:

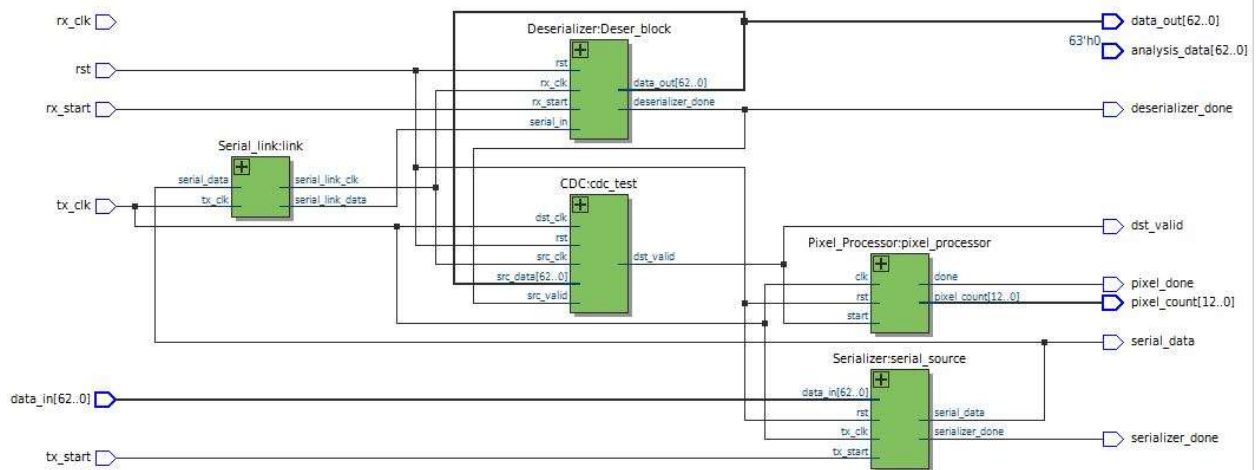
1. Function requirements

Requirement	Specification
Communication distance	5m
FPGA clock requirement	40MHz
Pixel per line	4096
Output width	63 bits
Processing Interval	56us/line
Ejectors Output	63
Communication	Serial
DIO lines	Minimum possible
Protocols	Time-synchronized
Future expansion	Required

2. Proposed Architecture



3. RTL Block Diagram



Serializer – converts the 63-bit parallel data into serial data.

Serial Link – represents the communication path between the two sides.

Deserializer – reconstructs the 63-bit parallel data.

CDC – transfers the data safely between asynchronous clock domains.

Pixel Processor – processes the 4096-pixel line and generates the 63-bit ejector command.

Handshake – ensures reliable transfer between clock domains.

4. RTL Implementation

4.1 Serialize

63-bit parallel input



shift register



serial_data

4.2 Deserializer

serial_data



shift register

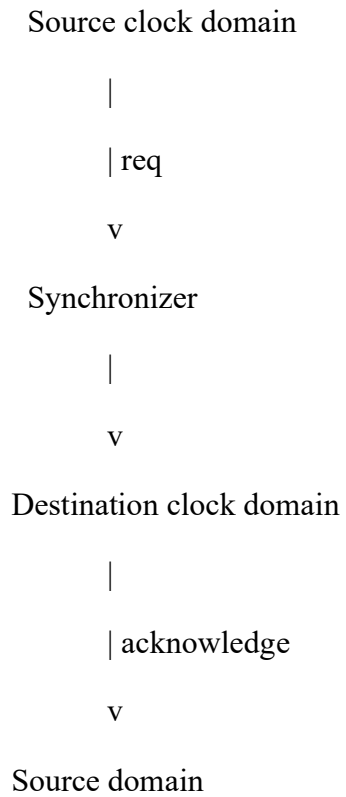


63-bit data_out

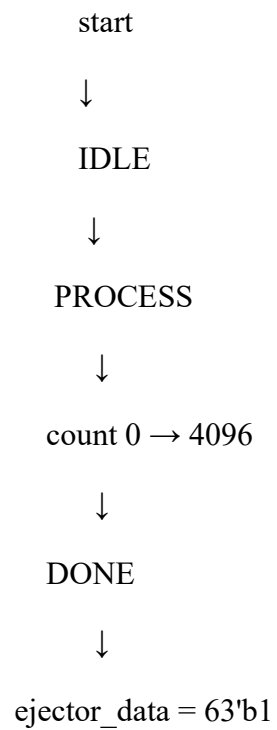


deserializer_done

4.3 CDC



4.4 Pixel Processor



```
count <= count + 13'd2;

pixel_count <= pixel_count + 13'd2;

and finally reaches:

pixel_count = 4096
```

5. Schematic

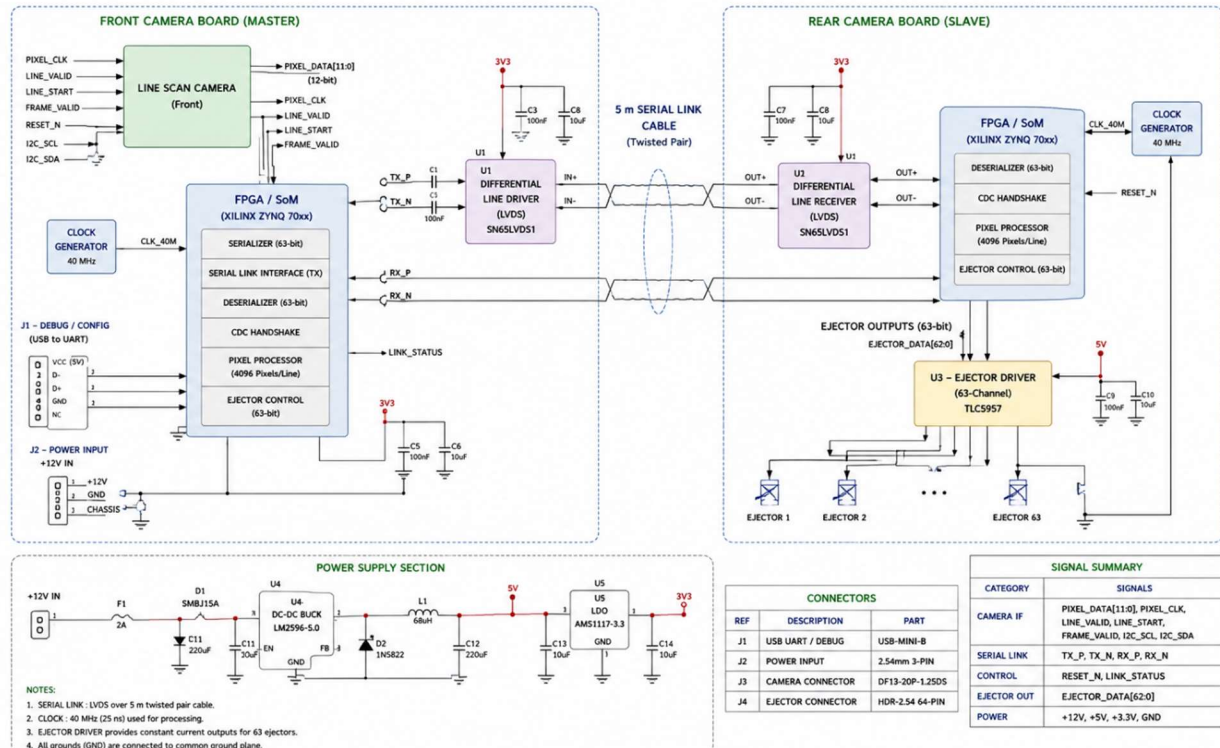


Figure : RTL-generated schematic of the Serial Link and CDC system

6. Simulation Environment / Datapath

Simulation Setup:

Simulator	Icarus Verilog
Waveform	VCD
TX Clock	10 ns / 100 MHz
RX Clock	14 ns / 71.43 MHz
Processing Clock	25 ns / 40 MHz
Transactions	4
Data width	63 bits
Reset	Applied initially

7.Simulation Datapath:

Testbench

|

| 63-bit test data

v

CynLr_Link_top

|

+--> Serializer

| |

| v

| Serial Data

| |

| v

| Deserializer

| |

| v

| 63-bit

| |

| v

| CDC

| |

| v

| Pixel Processor

| |

| +----> pixel_count = 4096

| |

| +----> ejector_data

| |

| +----> pixel_done

|

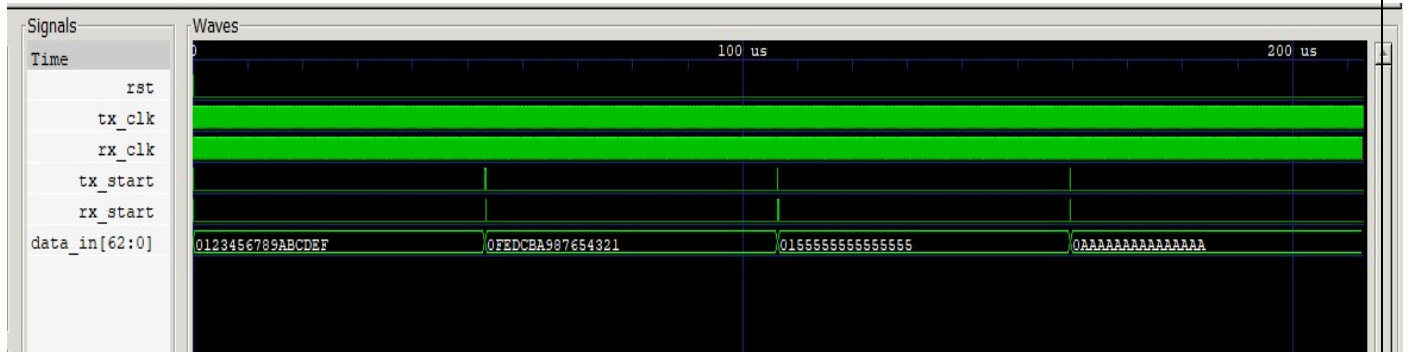
v

Testbench checks

The testbench provides a 63-bit input data word and start control. The data is serialized, transmitted through the serial link, deserialized, transferred through the CDC handshake, and finally supplied to the pixel processor. The testbench checks the data at each major stage.

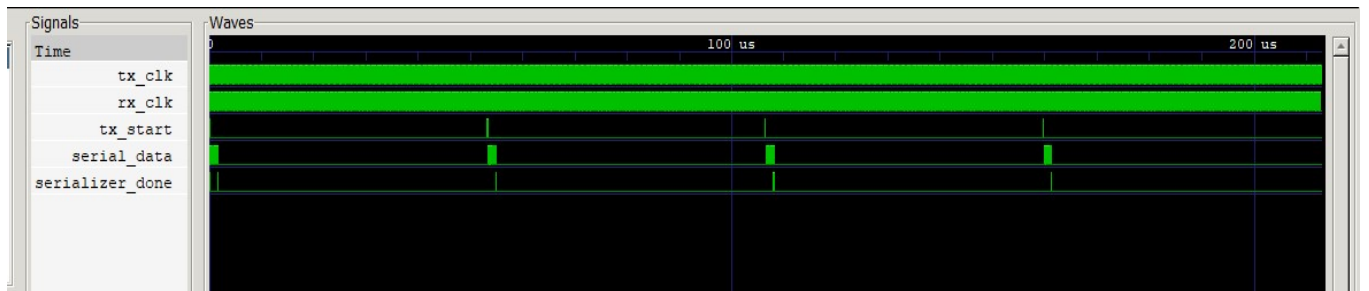
8. Waveform Verification

Waveform 1 — Reset and transaction start



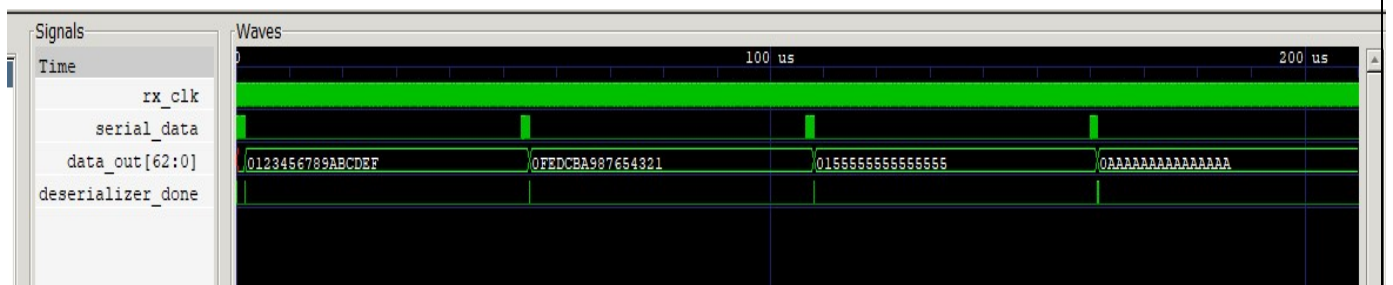
Purpose – Verify reset and transaction initiation

Waveform 2 — Serial transmission



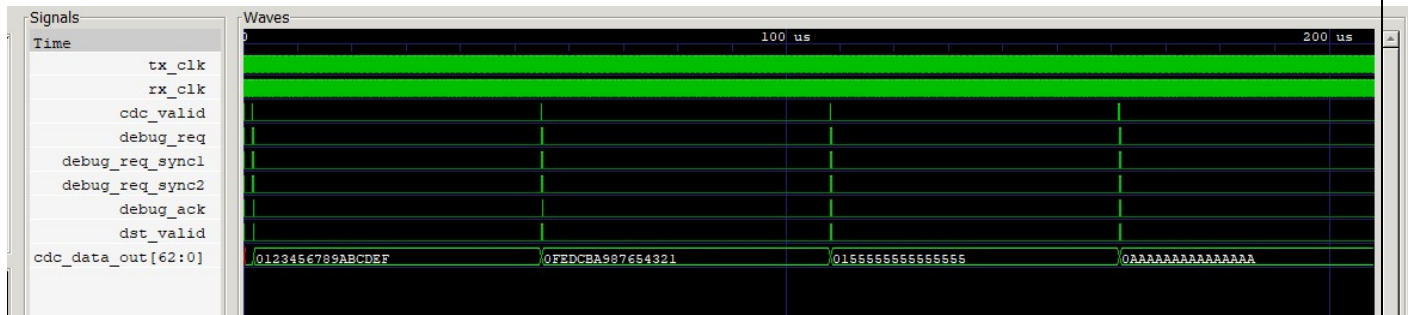
Purpose-Verify 63-bit data serialization

Waveform 3- Deserialization



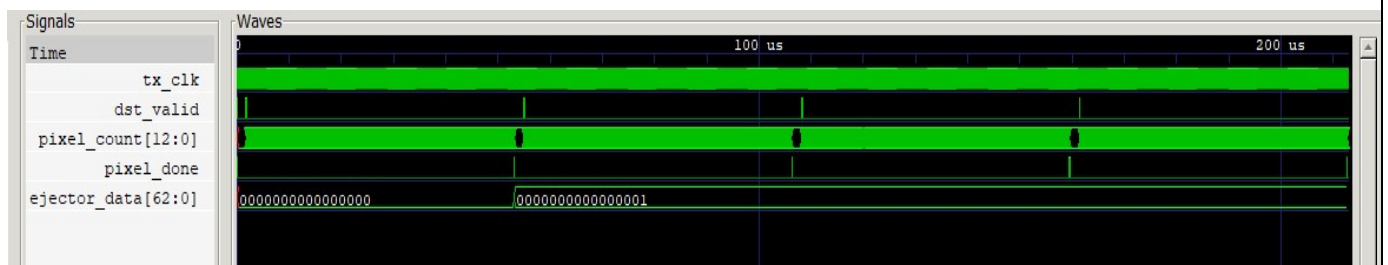
Purpose-Verify that the received serial data is reconstructed correctly.

Waveform 4- CDC

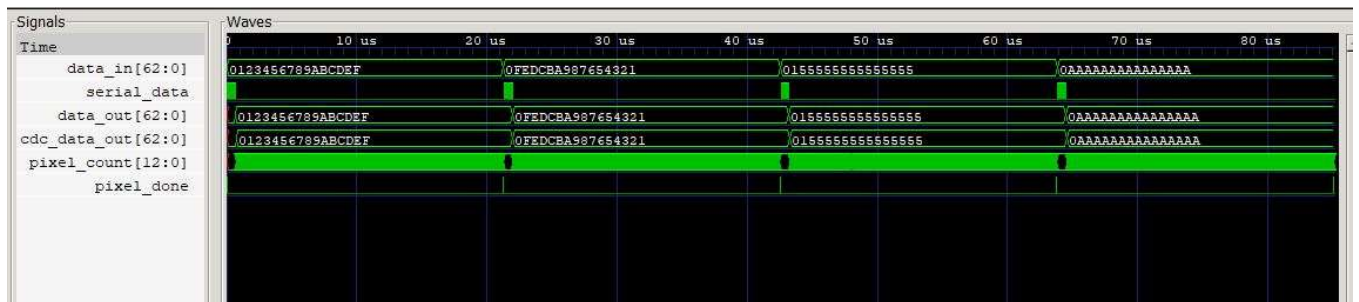


Purpose-verify safe transfer between asynchronous clock domain

Waveform 5- Pixel Processor



Waveform 6- Complete flow



9. 56us Requirement

Requirement:

The FPGA must process one complete line of 4096 pixel within 56us.

Parameter	Result
Pixels per line	4096
Clock frequency	40Mhz
Required processing time	$\leq 56\mu s$
Mearsured processing time	51.175us
Result	PASS

The simulation measured a processing time of 51.175us for processing 4096 pixels. Since 51.175us is less than the required maximum of 56us, the design satisfies the processing- time requirement.

```
=====
                        FINAL TEST SUMMARY
=====
TOTAL TRANSACTIONS = 4
TOTAL PASS         = 16
TOTAL FAIL         = 0
=====
FINAL RESULT : ALL TESTS PASSED
PIXEL PROCESSOR DONE = 212613000 ns
PROCESSING TIME      = 51175000 ns
56 us REQUIREMENT : PASS

TIME=212638000 : SIMULATION FINISHED
```

10. Simulation Results

```
=====
                        TRANSACTION 1 STARTED
=====
TIME=20000 : TEST DATA = 0123456789abcdef
TIME=25000 : TX_START = 1
TIME=50000 : TX_START = 0
TIME=75000 : RX_START = 1
TIME=100000 : WAITING FOR DESERIALIZER_DONE
TIME=1638000 : DESERIALIZER_DONE = 1
TIME=1638000 : DATA_OUT = 0123456789abcdef
SERIAL LINK : PASS
TIME=1638000 : WAITING FOR CDC_DST_VALID
TIME=1788000 : DST_VALID = 1
TIME=1788000 : CDC_DATA_OUT = 0123456789abcdef
CDC : PASS
TIME=1788000 : WAITING FOR PIXEL PROCESSOR
PIXEL PROCESSOR START = 1813000 ns
PIXEL PROCESSOR START = 1913000 ns
TIME=53063000 : PIXEL PROCESSOR DONE
PIXEL COUNT   = 4096
EJECTOR DATA = 0000000000000001
PIXEL PROCESSOR : PASS
PIXEL PROCESSOR : PASS

-----
TRANSACTION 1 COMPLETED
PASS COUNT = 4
FAIL COUNT = 0
-----
PIXEL PROCESSOR DONE = 53088000 ns
```

Transaction	Serial Link	CDC	Pixel Count	Pixel Processor	Processing Time
1	PASS	PASS	4096	PASS	51.175us
2	PASS	PASS	4096	PASS	51.175us
3	PASS	PASS	4096	PASS	51.175us
4	PASS	PASS	4096	PASS	51.175us

Verification	Results
Data width	63 bit PASS
Pixels/line	4096
Processing Clock	40 MHz PASS
Maximum processing time	51.175us <56 us
Transactions	4 PASS
Total checks	16
Total PASS	16
TOTAL FAIL	0

12.Final Verification Table

Requirement Verification

Requirement	Implementation/Measurement
63-bit data	63-bit input/output verified
4096 pixels/line	pixel count=4096
40 MHz Processing clock	25ns clock period
≤ 56 us processing	51.175us measured
Serial data transfer	4/4 transaction correct
CDC transfer	4/4 transaction correct
Ejector output	63'b1 observed
Overall simulation	16 PASS,0 FAIL

13. FPGA Implementation/QUARTUS Results

Flow Summary	
<<Filter>>	
Flow Status	Successful - Thu Aug 27 10:21:16 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	SerialPath
Top-level Entity Name	CynLr_Link_top
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	99 / 56,480 (< 1 %)
Total registers	238
Total pins	212 / 268 (79 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

14.Static timing Analysis

Slow 1100mV 85C Model Setup Summary			
<<Filter>>			
	Clock	Slack	End Point TNS
1	tx_clk	14.982	0.000

Slow 1100mV 85C Model Hold Summary			
<<Filter>>			
	Clock	Slack	End Point TNS
1	tx_clk	0.361	0.000

14. One important correction to your project story

I developed an FPGA RTL communication and synchronization path for a camera/ejector system, where 63-bit line-level decisions need to be transferred between camera domains under tight timing and limited I/O constraints.

15.Future Work

1. Add physical serial-channel modeling
2. Improve pixel-processing functionality
3. Add assertions
4. Extend constrained-random verification

Appendix A — RTL Source Files

A.1 CynLr_Link_top.v

```
`timescale 1ns/1ps
module CynLr_Link_top(
    input rst,
    input tx_clk,
    input rx_clk,
    input tx_start,
    input rx_start,
    input [62:0] data_in,

    output wire serial_data,
    output wire [62:0] data_out,
    output wire serializer_done,
    output wire deserializer_done,

    output wire [62:0]cdc_data_out,
    output wire dst_valid,
    //
    output wire debug_req,
    output wire debug_ack,
    output wire debug_req_sync1,
    output wire debug_req_sync2,
    //
    output wire [62:0] ejector_data,
    output wire [12:0] pixel_count,
    output wire pixel_done
);
```

```

//=====
// INTERNAL SIGNALS
//=====

wire serial_link_data;
wire serial_link_clk;
wire cdc_valid;
assign cdc_valid=deserializer_done;

//=====
// SERIALIZER
//=====

Serializer serial_source(
    .rst(rst),
    .tx_clk(tx_clk),
    .tx_start(tx_start),
    .data_in(data_in),
    .serial_data(serial_data),
    .serializer_done(serializer_done)
);

//=====
// SERIAL LINK
//=====

Serial_link link(
    .serial_data(serial_data),
    .tx_clk(tx_clk),
    .serial_link_data(serial_link_data),
    .serial_link_clk(serial_link_clk)
);

//=====
// DESERIALIZER
//=====

Deserializer Deser_block(
    .rst(rst),
    .rx_clk(serial_link_clk),
    .serial_in(serial_link_data),
    .rx_start(rx_start),
    .data_out(data_out),

```

```

        .deserializer_done(deserializer_done)
    );

    CDC cdc_test (
        .rst      (rst),
        .src_clk   (serial_link_clk),
        .dst_clk   (tx_clk),
        .src_data  (data_out),
        .src_valid (cdc_valid),

        .dst_data  (cdc_data_out),
        .dst_valid (dst_valid),
        //
        .debug_req  (debug_req),
        .debug_ack  (debug_ack),
        .debug_req_sync1 (debug_req_sync1),
        .debug_req_sync2 (debug_req_sync2)
    );

    //=====
    // PIXEL PROCESSOR
    //=====

    Pixel_Processor pixel_processor (
        .clk      (tx_clk),
        .rst      (rst),
        .start     (dst_valid),
        .done      (pixel_done),
        .ejector_data (ejector_data),
        .pixel_count (pixel_count)
    );
endmodule

```

A.2 Serializer.v

```

`timescale 1ns/1ps
module Serializer(rst,tx_clk,tx_start,serializer_done,data_in,serial_data);
input rst,tx_clk,tx_start;
input [62:0]data_in;
output reg serial_data;
output reg serializer_done;
reg [62:0]data_reg;
reg [1:0]state;
reg [6:0]count;

```

```

parameter IDLE_STATE=2'b00;
parameter LOAD_STATE=2'b01;
parameter SHIFT_STATE=2'b10;
always@(posedge tx_clk) begin
    if (rst) begin
        serial_data<=1'b0;
        data_reg<=63'b0;
        state<=IDLE_STATE;
        serializer_done<=1'b0;
        count<=7'd0;
    end
    else begin
        case(state)
            //=====
            // IDLE STATE
            //=====
            IDLE_STATE: begin
                serializer_done<=1'b0;
                serial_data<=1'b0;
                count<=7'd0;
                if(tx_start)
                    state<=LOAD_STATE;
                else
                    state<=IDLE_STATE;
            end
            //=====
            // LOAD STATE
            //=====
            LOAD_STATE:begin
                data_reg<=data_in;
                serial_data <= data_in[62];
                count<=7'd0;
                state<=SHIFT_STATE;
            end
            //=====
            // SHIFT STATE
            //=====
            SHIFT_STATE:begin
                // Output next bit
                serial_data <= data_reg[61];
                // Shift register
                data_reg<={data_reg[61:0],1'b0};
                if (count==7'd61)begin
                    count<=7'd62;
                    state<=IDLE_STATE;
                end
            end
        endcase
    end
end

```

```

        serializer_done<=1'b1;
    end
    else begin
        count<=count+7'd1;
        state<=SHIFT_STATE;
    end
end
end
//=====
// DEFAULT CASE
//=====
default:begin
    state<=IDLE_STATE;
    serial_data<=1'b0;
    data_reg<=63'b0;
    serializer_done<=1'b0;
    count<=7'd0;
end

endcase
end
end
endmodule

```

A.3 Serial_link.v

```

`timescale 1ns/1ps
module Serial_link(serial_data,serial_link_data,tx_clk,serial_link_clk);
input serial_data;
input tx_clk;
output serial_link_data;
output serial_link_clk;
assign serial_link_data=serial_data;
assign serial_link_clk=tx_clk;
endmodule

```

A.4 Deserializer.v

```

module Deserializer(rst,rx_clk,rx_start,serial_in,deserializer_done,data_out);
input rst,rx_clk,rx_start;
input serial_in;
output reg [62:0] data_out;
output reg deserializer_done;
reg [1:0]state;
reg [6:0]count;
reg [62:0]data_reg;

```

```

parameter IDLE_STATE=2'b00;
parameter RECEIVE_STATE=2'b01;
parameter FINAL_STATE=2'b10;
always @(posedge rx_clk)begin
    if(rst)begin
        data_out<=63'b0;
        data_reg<=63'b0;
        deserializer_done<=1'b0;
        count<=7'd0;
        state<=IDLE_STATE;
    end
    else begin
        case(state)
            //=====
            // IDLE STATE
            //=====
            IDLE_STATE:begin
                deserializer_done<=1'b0;
                count<=7'd0;
                if(rx_start)begin
                    // Capture FIRST serial bit immediately
                    data_reg <= {62'b0, serial_in};
                    // First bit captured
                    count <= 7'd1;
                    state<=RECEIVE_STATE;
                end
            else
                state<=IDLE_STATE;
            end
            //=====
            // RECEIVER STATE
            //=====
            RECEIVE_STATE:begin
                if(count==7'd62)begin
                    data_out<={data_reg[61:0], serial_in};
                    data_reg<={data_reg[61:0],serial_in};
                    count<=7'd63;
                    deserializer_done<=1'b1;
                    state<=FINAL_STATE;
                end
            else begin
                data_reg <= {data_reg[61:0], serial_in};
                count<=count+7'd1;
                state<=RECEIVE_STATE;
            end
        end
    end
end

```

```

end
//=====
//FINAL STATE
//=====
FINAL_STATE:begin
    deserializer_done<=1'b0;
    state<=IDLE_STATE;
end
//=====
//DEFAULT CASE
//=====
default: begin
    data_out<=63'b0;
    data_reg<=63'b0;
    count<=7'd0;
    deserializer_done<=1'b0;
    state<=IDLE_STATE;
end
endcase
end
end
endmodule

```

A.5 CDC.v

```

`timescale 1ns/1ps
/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////HANDSHAKE SOURCE/////////////////////////////////////////////////////////////////
/////////////////////////////////////////////////////////////////
module Handshake source(rst,src_data,src_clk,src_valid,ack_src,req_src,src_data out);
input rst,src_clk,src_valid,ack_src;
input [62:0]src_data;
output reg req_src;
output reg [62:0]src_data out;
reg [1:0]state;
reg ack_sync ff1;
reg ack_sync ff2;
//reg [62:0]src_data reg;
parameter IDLE_STATE=2'b00;
parameter SEND_REQ_STATE=2'b01;
parameter WAIT_ACK_STATE=2'b10;
always @(posedge src_clk) begin
    if(rst)begin
        req_src<=1'b0;

```

```

state<=IDLE_STATE;
ack_sync_ff1<=1'b0;
ack_sync_ff2<=1'b0;
//src_data_reg<=63'b0;
src_data_out<=63'b0;
end
else begin
    //Synchronization ACK from destination
    ack_sync_ff1<=ack_src;
    ack_sync_ff2<=ack_sync_ff1;
case (state)
//=====
// IDLE STATE
//=====
IDLE_STATE:begin
    req_src<=1'b0;
    if(src_valid)
        state<=SEND_REQ_STATE;
    else
        state<=IDLE_STATE;
end
//=====
// SEND REQUEST
//=====
SEND_REQ_STATE: begin
    req_src<=1'b1;
    //src_data_reg<=src_data;
    src_data_out<=src_data;
    state<=WAIT_ACK_STATE;
end
//=====
// WAIT ACK REQUEST
//=====
WAIT_ACK_STATE:begin
    req_src<=1'b1;
    if(ack_sync_ff2)begin
        req_src<=1'b0;
        state<=IDLE_STATE;
    end
    else begin
        state<=WAIT_ACK_STATE;
    end
end
//=====
// DEFAULT STATE

```

```

//=====
default: begin
    req_src<=1'b0;
    state<=IDLE_STATE;
end
endcase
end
end
endmodule

////////////////////////////////////
////////// HANDSHAKE DESTINATION////////////////////////////////////
////////////////////////////////////

module Handshake_Destination(rst,dst_clk,ack_dst,req_dst,data_in,dst_data,dst_valid);
input rst,dst_clk,req_dst;
input [62:0]data_in;
output reg ack_dst;
output reg [62:0]dst_data;
output reg dst_valid;

reg [1:0]state;
reg req_sync_ff1;
reg req_sync_ff2;

parameter IDLE_STATE=2'b00;
parameter WAIT_REQ=2'b01;
parameter CAPTURE_STATE=2'b10;
parameter SEND_ACK=2'b11;
always @(posedge dst_clk)begin
    if(rst)begin
        state<=IDLE_STATE;
        req_sync_ff1<=1'b0;
        req_sync_ff2<=1'b0;
        ack_dst<=1'b0;
        dst_data<=63'b0;
        dst_valid<=1'b0;
    end
    else begin
        // dst_valid is a one-clock pulse
        dst_valid<=1'b0;
        //Synchronization REQ from source
        req_sync_ff1<=req_dst;
        req_sync_ff2<=req_sync_ff1;
        case(state)
            //=====

```

```

// IDLE STATE
//=====
IDLE_STATE:begin
    ack_dst<=1'b0;
    state<=WAIT_REQ;
end
//=====
// WAIT REQUEST
//=====
WAIT_REQ: begin
    ack_dst<=1'b0;
    if (req_sync_ff2)
        state<=CAPTURE_STATE;
    else
        state<=WAIT_REQ;
    end
end
//=====
// CAPTURE STATE
//=====
CAPTURE_STATE:begin
    dst_data<=data_in;
    dst_valid<=1'b1;
    ack_dst<=1'b0;
    state<=SEND_ACK;
end
//=====
// SEND ACK
//=====
SEND_ACK:begin
    ack_dst<=1'b1;
    state<=IDLE_STATE;
end
//=====
// Default state
//=====
default:begin
    ack_dst<=1'b0;
    dst_data<=63'b0;
    dst_valid<=1'b0;
    state<=IDLE_STATE;
end
endcase
end
endmodule

```

```

////////////////////////////////////
//////// CDC Module////////////////////////////////////
////////////////////////////////////

module CDC (
    input rst,
    input src_clk,
    input dst_clk,
    input [62:0] src_data,
    input src_valid,

    output wire [62:0] dst_data,
    output wire dst_valid,
    // DEBUG SIGNALS
    output wire debug_req,
    output wire debug_ack,
    output wire debug_req_sync1,
    output wire debug_req_sync2
);

    // wires
    wire req_src;
    wire ack_dst;
    wire [62:0] src_data_out;
    wire [62:0] dst_data_int;
    wire dst_valid_int;
    //
    assign debug_req    = req_src;
    assign debug_ack    = ack_dst;
    assign debug_req_sync1 = u_destination.req_sync_ff1;
    assign debug_req_sync2 = u_destination.req_sync_ff2;

    //=====
    // SOURCE
    //=====

    Handshake_source u_source (
        .rst      (rst),
        .src_clk   (src_clk),
        .src_valid (src_valid),
        .src_data  (src_data),
        .ack_src   (ack_dst),
        .req_src   (req_src),
        .src_data_out (src_data_out)

```

```

);
assign dst_data = dst_data_int;
assign dst_valid = dst_valid_int;

//=====
// DESTINATION
//=====

Handshake_Destination u_destination (
    .rst    (rst),
    .dst_clk (dst_clk),
    .ack_dst (ack_dst),
    .req_dst (req_src),
    .data_in (src_data_out),
    .dst_data (dst_data_int),
    .dst_valid (dst_valid_int)
);

endmodule

```

A.6 Pixel_Processor.v

```

module Pixel_Processor(clk,rst,start,done,ejector_data,pixel_count);
input clk,rst,start;
output reg done;
output reg [62:0]ejector_data;
output reg [12:0]pixel_count;
reg [1:0]state;
reg [12:0]count;
parameter
IDLE_STATE=2'b00,
PROCESS_STATE=2'b01,
DONE_STATE=2'b10;
always @(posedge clk) begin
    if(rst)begin
        done<=1'b0;
        ejector_data<=63'b0;
        pixel_count<=13'd0;
        count<=13'd0;
        state<=IDLE_STATE;
    end
    else begin
        // Default: DONE is a one-clock pulse
        done <= 1'b0;

```

```

case(state)
//=====
// UDLE STATE
//=====
IDLE_STATE:begin
    if(start)begin
        count <= 13'd0;
        pixel_count<=13'd0;
        state<=PROCESS_STATE;
    end
    else
        state<=IDLE_STATE;
    end
//=====
// PROCESS STATE
//=====
PROCESS_STATE:begin
    if (count>=13'd4096)begin
        count<=13'd4096;
        pixel_count<=13'd4096;
        state<=DONE_STATE;
    end
    else begin
        count<=count+13'd2;
        pixel_count<=pixel_count+13'd2;
        state<=PROCESS_STATE;
    end
end
end
//=====
// DONE STATE
//=====
DONE_STATE:begin
    ejector_data<=63'b1;
    done<=1'b1;
    state<=IDLE STATE;
end
//=====
//DEFAULT STATE
//=====
default:begin
    done<=1'b0;
    pixel_count<=13'd0;
    ejector data<=63'b0;
    count<=13'b0;
    state<=IDLE_STATE;
end

```

```

        end
    endcase
end
end
endmodule

```

A.7 CynLr_Link_top_tv.v

```

`timescale 1ns/1ps

module tb;

//=====
// TESTBENCH SIGNALS
//=====

reg rst;

reg tx_clk;
reg rx_clk;

reg tx_start;
reg rx_start;

reg [62:0] data_in;
reg pixel_clk;

wire serial_data;
wire [62:0] data_out;

wire serializer_done;
wire deserializer_done;

wire [62:0] cdc_data_out;
wire dst_valid;
wire cdc_valid;

wire pixel_done;
wire [62:0] ejector_data;
wire [12:0] pixel_count;

wire debug_req;
wire debug_ack;
wire debug_req_sync1;
wire debug_req_sync2;

```

```

assign cdc_valid = deserializer_done;

//=====
// TEST COUNTERS
//=====

integer pass_count;
integer fail_count;
integer transaction_count;

//=====
// DUT
// CONNECTION NOT CHANGED
//=====

CynLr_Link_top dut (
    .rst            (rst),
    .tx_clk         (tx_clk),
    .rx_clk         (rx_clk),
    .tx_start       (tx_start),
    .rx_start       (rx_start),
    .data_in        (data_in),

    .serial_data    (serial_data),
    .data_out       (data_out),

    .serializer_done (serializer_done),
    .deserializer_done (deserializer_done),

    .cdc_data_out   (cdc_data_out),
    .dst_valid      (dst_valid),

    .debug_req      (debug_req),
    .debug_ack      (debug_ack),
    .debug_req_sync1 (debug_req_sync1),
    .debug_req_sync2 (debug_req_sync2),
    .ejector_data(ejector_data),
    .pixel_count(pixel_count),
    .pixel_done(pixel_done)
);

```

```

//=====
// TX CLOCK
// HALF PERIOD = 5 ns
// PERIOD    = 10 ns
//=====

initial begin

    tx_clk = 1'b0;

    forever #12.5 tx_clk = ~tx_clk;

end

//=====
// RX CLOCK
// HALF PERIOD = 7 ns
// PERIOD    = 14 ns
//=====

initial begin

    rx_clk = 1'b0;

    forever #7 rx_clk = ~rx_clk;

end

//=====
// TRANSACTION TASK
//=====

task automatic send_transaction;

    input [62:0] test_data;

begin

    transaction_count = transaction_count + 1;

    $display("");
    $display("=====");
    $display("    TRANSACTION %0d STARTED",

```

```

        transaction_count);
$display("=====");

$display("TIME=%0t : TEST DATA = %h",
        $time,
        test_data);

//=====
// STEP 1 : SEND DATA TO SERIALIZER
//=====

@(negedge tx_clk);

data_in = test_data;
tx_start = 1'b1;

$display("TIME=%0t : TX_START = 1",
        $time);

@(negedge tx_clk);

tx_start = 1'b0;

$display("TIME=%0t : TX_START = 0",
        $time);

//=====
// STEP 2 : START DESERIALIZER
//=====

@(negedge dut.serial_link_clk);

rx_start = 1'b1;

$display("TIME=%0t : RX_START = 1",
        $time);

@(negedge dut.serial_link_clk);

rx_start = 1'b0;

//=====

```

```

// STEP 3 : WAIT FOR DESERIALIZER
//=====

$display("TIME=%0t : WAITING FOR DESERIALIZER_DONE",
    $time);

@(posedge deserializer done);

$display("TIME=%0t : DESERIALIZER_DONE = 1",
    $time);

$display("TIME=%0t : DATA_OUT = %h",
    $time,
    data_out);

//=====
// STEP 4 : SERIAL LINK CHECK
//=====

if (data_out == test_data) begin

    $display("SERIAL LINK : PASS");

    pass_count = pass_count + 1;

end
else begin

    $display("SERIAL LINK : FAIL");

    $display("EXPECTED = %h",
        test_data);

    $display("ACTUAL   = %h",
        data_out);

    fail_count = fail_count + 1;

end

//=====
// WAIT CDC DESTINATION
//=====

```

```

$display("TIME=%0t : WAITING FOR CDC DST_VALID",
    $time);

// Wait for CDC destination
@(posedge dst_valid);

$display("TIME=%0t : DST_VALID = 1",
    $time);

$display("TIME=%0t : CDC DATA OUT = %h",
    $time, cdc_data_out);

//=====
// CDC CHECK
//=====

if (cdc_data_out == test_data) begin

    $display("CDC : PASS");
    pass_count = pass_count + 1;

end
else begin

    $display("CDC : FAIL");
    $display("EXPECTED = %h", test_data);
    $display("ACTUAL   = %h", cdc_data_out);

    fail_count = fail_count + 1;

end

//=====
// WAIT FOR PIXEL PROCESSOR
//=====

$display("TIME=%0t : WAITING FOR PIXEL PROCESSOR",
    $time);

@(posedge pixel_done);

$display("TIME=%0t : PIXEL PROCESSOR DONE",
    $time);

```

```

$display("PIXEL COUNT  = %0d", pixel_count);
$display("EJECTOR DATA  = %h", ejector_data);

if ((pixel_count == 13'd4096) &&
    (ejector_data == 63'b1)) begin

    $display("PIXEL PROCESSOR : PASS");
    pass_count = pass_count + 1;

end
else begin

    $display("PIXEL PROCESSOR : FAIL");
    $display("EXPECTED COUNT  = 4096");
    $display("ACTUAL COUNT   = %0d", pixel_count);
    $display("EXPECTED EJECTOR = 1");
    $display("ACTUAL EJECTOR  = %h",
            ejector_data);

    fail_count = fail_count + 1;

end
//////////
if ((pixel_count == 13'd4096) &&
    (ejector_data == 63'd1)) begin

    $display("PIXEL PROCESSOR : PASS");
    pass_count = pass_count + 1;

end
else begin

    $display("PIXEL PROCESSOR : FAIL");
    fail_count = fail_count + 1;

end

//=====
// TRANSACTION RESULT
//=====

$display("");
$display("-----");
$display("TRANSACTION %0d COMPLETED",
        transaction_count);

```

```

$display("PASS COUNT = %0d",
        pass_count);

$display("FAIL COUNT = %0d",
        fail_count);

$display("-----");

end

endtask

//=====
// MAIN TEST
//=====

initial begin

    //=====
    // WAVEFORM
    //=====

    $dumpfile("wave.vcd");

    $dumpvars(0, tb);

    //=====
    // INITIAL VALUES
    //=====

    rst    = 1'b1;

    tx_start = 1'b0;

    rx_start = 1'b0;

    data_in = 63'b0;

    pass_count    = 0;
    fail_count    = 0;
    transaction_count = 0;

```

```

//=====
// TEST START
//=====

$display("");
$display("=====");
$display("    SERIAL LINK + CDC TESTBENCH");
$display("=====");

$display("TX CLOCK PERIOD = 10 ns");

$display("RX CLOCK PERIOD = 14 ns");

$display("TOTAL TRANSACTIONS = 4");

$display("=====");

//=====
// RESET
//=====

#20;

rst = 1'b0;

$display("");
$display("TIME=%0t : RESET RELEASED",
    $time);

//=====
// TRANSACTION 1
//=====

send_transaction(
    63'h123456789ABCDEF
);

//=====
// GAP
//=====

```

```

repeat (5)
    @(negedge tx_clk);

//=====
// TRANSACTION 2
//=====

send_transaction(
    63'h0FEDCBA987654321
);

//=====
// GAP
//=====

repeat (5)
    @(negedge tx_clk);

//=====
// TRANSACTION 3
//=====

send_transaction(
    63'h1555555555555555
);

//=====
// GAP
//=====

repeat (5)
    @(negedge tx_clk);

//=====
// TRANSACTION 4
//=====

send_transaction(
    63'hAAAAAAAAAAAAAAAA
);

```

```

//=====
// FINAL SUMMARY
//=====

$display("");
$display("=====");
$display("          FINAL TEST SUMMARY");
$display("=====");

$display("TOTAL TRANSACTIONS = %0d",
        transaction_count);

$display("TOTAL PASS      = %0d",
        pass_count);

$display("TOTAL FAIL      = %0d",
        fail_count);

$display("=====");

if (fail_count == 0) begin

    $display("FINAL RESULT : ALL TESTS PASSED");

end
else begin

    $display("FINAL RESULT : TEST FAILED");

end

//=====
// END SIMULATION
//=====

#50;

$display("");
$display("TIME=%0t : SIMULATION FINISHED",
        $time);

```

```

$finish;

end
///
time start_time;
time done_time;

always @(posedge tx_clk) begin
    if (dst_valid) begin
        start_time = $time;
        $display("PIXEL PROCESSOR START = %0t ns", $time);
    end

    if (pixel_done) begin
        done_time = $time;
        $display("PIXEL PROCESSOR DONE = %0t ns", $time);
        $display("PROCESSING TIME    = %0t ns", done_time - start_time);

        if ((done_time - start_time) <= 56000)
            $display("56 us REQUIREMENT : PASS");
        else
            $display("56 us REQUIREMENT : FAIL");
        end
    end
end
endmodule

```